

CS355: Programming Paradigms Lab (Autumn 2025)

Lab Midsem Exam (2 hours, 40 marks)

Instructions:

- Grievances will be considered only if you act like an imperative program (follow all the instructions).
- First of all create a folder named `submission_RollNum` (where `RollNum` is your roll number in caps) on the Desktop. All your solutions should be inside this folder.
- The solutions to Q1..Q5 should be in files named `Q1.scm`..`Q5.scm`. The solutions to Q6 and Q7 should be in files named `Q6.java` and `Q7.java`, respectively (no other files).
- The first line in each Scheme file should be `# lang sicp`.
- For the Scheme questions, **delete** any testing code before making your submission.
- For the Java questions, none of the classes in your submission should be marked `public`. Do not use any packages.
- For checking your solutions, run the script `validate.py` present on your desktop. This will check your solutions for correctness on the testcase present in the question paper (passing this step would fetch you at least one mark per question).
- When you are done with all the questions, call a TA. They would run a `check` script and a `submit` script on your system using a password. This will zip the submission and transfer it to our submission server. Leave the room only when the TA confirms that this step was successful.
- For final evaluation, we would add a few more checks to `validate.py` and a few more testcases, and award you marks.

Q1 [5]. Define a procedure `flatten` that takes a nested list and returns a flat list of all elements (in left-to-right order).

To check your code, we would paste calls to `flatten` below your code, which should produce the expected output as shown.

```
> (flatten '((1 2) (3 (4 5))))  
(1 2 3 4 5)
```

Q2 [5] Define a procedure `apply-n` that takes a function `f`, an integer `n`, and a value `x`, and applies `f` to `x` exactly `n` times.

To check your code, we would paste calls to `apply-n` below your code, which should produce the expected output as shown.

```
> (apply-n (lambda (y) (* y 2)) 3 1)  
8
```

Q3 [6] Write a procedure `primes-under` that, given a number `n`, returns all prime numbers less than `n`. You must use `filter` (paste its definition before writing your code from the file `filter.scm` and let it remain in your solution).

To check your code, we would paste calls to `primes-n` below your code, which should produce the expected output as shown.

```
> (primes-under 10)
(2 3 5 7)
```

Q4 [6] Write a procedure `running-max` that returns a list of the running maximum of a sequence of numbers. Use `foldl` (paste its definition before writing your code from the file `foldl.scm` and let it remain in your solution).

To check your code, we would paste calls to `running-max` below your code, which should produce the expected output as shown.

```
> (running-max '(3 1 4 2 6))
(3 3 4 4 6)
```

Q5 [6] Define a procedure `words-longer-than` that takes a list of words and an integer `k`, and returns all words whose length is greater than `k`, sorted alphabetically.

To check your code, we would paste calls to `words-longer-than` below your code, which should produce the expected output as shown.

```
> (words-longer-than '("the" "quick" "brown" "fox" "jumps") 3)
("brown" "jumps" "quick")
```

Q6 [6] Define an abstract class `Shape` with:

- Abstract methods `double area()` and `double perimeter()`.
- A method `toString()` that prints "<Shape> with area = <area>, perimeter = <perimeter>" using these methods.

Create `Circle` and `Rectangle` subclasses. Write a static method `largestShape` in a class named `ShapeUtils` that takes a `List<Shape>` and returns the shape with the largest area.

To check your code, we would create a main class and add code like the following in its main method. When we run it, we should get the expected output as shown.

```
List<Shape> shapes = Arrays.asList(new Circle(2), new Rectangle(3, 5));
System.out.println(ShapeUtils.largestShape(shapes)); // should print
"Rectangle with area = 15"
```

Q7 [6] Define an interface **Playable** with a method **int duration()**.

- **Song** implements **Playable** (has a fixed duration in seconds).
- **Playlist** implements **Playable** and stores a list of **Playable** items. Its duration is the sum of durations of all items.

To check your code, we would create a main class and add code like the following in its main method. When we run it, we should get the expected output as shown.

```
Playable song1 = new Song(200);
Playable song2 = new Song(180);
Playable playlist = new Playlist(Arrays.asList(song1, song2));
System.out.println(playlist.duration()); // should print 380
```